

COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS, PHYSICS AND INFORMATICS

FINDING THE LONGEST INDUCED CYCLES IN
CUBIC GRAPHS
BACHELOR THESIS

2025
ELIŠKA KRAJČIOVÁ

COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS, PHYSICS AND INFORMATICS

FINDING THE LONGEST INDUCED CYCLES IN
CUBIC GRAPHS

BACHELOR THESIS

Study Programme: Computer Science
Field of Study: Computer Science
Department: Department of Computer Science
Supervisor: Mgr. Matúš Matok

Bratislava, 2025
Eliška Krajčiová



Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Eliška Krajčiová
Študijný program: informatika (Jednoodborové štúdium, bakalársky I. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: bakalárska
Jazyk záverečnej práce: anglický
Sekundárny jazyk: slovenský

Názov: Finding the longest induced cycles in cubic graphs
Hľadanie najdlhšej indukovanej kružnice v kubických grafoch

Anotácia: Problém hľadania najväčšieho indukovaného r -regulárneho podgrafu pre daný graf je problémom študovaným už od čias Erdős-a. Od tej doby bolo dokázané, že ide o NP-ťažký problém, avšak pre špecifické triedy grafov boli vyvinuté efektívne algoritmy. My sa budeme zaoberať iba indukovanými 2-regulárnymi podgrafmi, teda indukovanými kružnicami, v kubických grafoch. Aj keď sú známe dolné a horné ohraničenia na veľkosť indukovaných kružníc, efektívny algoritmus na ich hľadanie by mohol dopomôcť k zúženiu ohraničení pre triedu kubických grafov.

Cieľ: Cieľom práce je navrhnúť efektívny algoritmus na hľadanie najdlhšej indukovanej kružnice v kubických grafoch.

Kľúčové slová: Indukovaná kružnica, kubický graf, algoritmus

Vedúci: Mgr. Matúš Matok
Katedra: FMFI.KI - Katedra informatiky
Vedúci katedry: prof. RNDr. Martin Škoviera, PhD.

Spôsob sprístupnenia elektronickej verzie práce:
dočasne neprístupná, po uplynutí bez obmedzenia

Dátum zadania: 06.10.2024

Dátum schválenia: 23.10.2024

doc. RNDr. Dana Pardubská, CSc.
garant študijného programu

.....
študent

.....
vedúci práce



THESIS ASSIGNMENT

Name and Surname: Eliška Krajčiová
Study programme: Computer Science (Single degree study, bachelor I. deg., full time form)
Field of Study: Computer Science
Type of Thesis: Bachelor's thesis
Language of Thesis: English
Secondary language: Slovak

Title: Finding the longest induced cycles in cubic graphs

Annotation: The problem of finding the largest induced r -regular subgraph of a given graph is a problem studied since the dates of Erdős. It has been shown since, that it is in fact an NP-hard problem. Efficient algorithms were established for special graph classes. We are strictly interested in induced 2-regular subgraphs, i. e., induced cycles in cubic graphs. Although, some lower and upper bounds are known, an efficient algorithm might contribute to narrowing down the gap between these bounds, as they are not tight for cubic graphs.

Aim: The goal is to devise an efficient algorithm capable of finding the longest induced cycle of a cubic graph.

Keywords: Induced cycle, cubic graph, algorithm

Supervisor: Mgr. Matúš Matok
Department: FMFI.KI - Department of Computer Science
Head of department: prof. RNDr. Martin Škoviera, PhD.

Electronic version available:
dočasne neprístupná, po uplynutí bez obmedzenia

Assigned: 06.10.2024

Approved: 23.10.2024

doc. RNDr. Dana Pardubská, CSc.
Guarantor of Study Programme

Student

Supervisor

Acknowledgments: I wish to express my deepest gratitude to my supervisor, for his invaluable guidance, constant support, and for always making time to discuss and improve this work.

Abstrakt

Zaoberáme sa problémom hľadania najdlhšej indukovanej kružnice v kubických grafoch, špecializáciou všeobecného problému indukovaných cyklov. Vychádzajúc z extrémneho ohraničenia – každý kubický graf obsahuje indukovaný 2-regulárny podgraf s najviac $\frac{3n}{4}$ vrcholmi – navrhujeme spätné vyhľadávanie, ktoré postupne rozširuje cesty bez chord a uzatvára ich do kružníc. Hlavné heuristiky orezávania – rozšírenie, keď existuje len jedna voľba, vyradenie spracovaných vrcholov a včasné zastavenie, ak už nemožno nájsť lepší cyklus – významne znižujú vyhľadávací priestor. Pre grafy do 28 vrcholov naša metóda beží v milisekundách namiesto minút pre naívnú enumeráciu. Experimenty na rôznych triedach kubických grafov potvrdili správnosť a odhalili prípady s veľmi krátkymi najdlhšími kružnicami. Naše výsledky prinášajú štrukturálne poznatky aj praktický nástroj na skúmanie extrémnych cyklov v grafoch s nízkym stupňom.

Kľúčové slová: indukovaná kružnica, kubický graf, spätné vyhľadávanie, heuristiky orezávania, výpočtové experimenty

Abstract

We address the problem of finding the length of the longest induced cycle in cubic graphs, a specialized case of the broader induced-cycles problem. Leveraging known bounds—that any cubic graph contains an induced 2-regular subgraph on at most $\frac{3n}{4}$ vertices—we develop a backtracking algorithm that incrementally builds chord-free paths and closes them into cycles. Key pruning ideas—forcing single-choice extensions, discarding vertices once their best cycles are known, and abandoning paths that cannot surpass the current best—shrink the search dramatically. On graphs up to 28 vertices, our approach runs in milliseconds versus minutes for naïve subset-enumeration. Experiments on various cubic graph families confirm correctness and illustrate extremal examples with very short longest induced cycles. Our results combine new structural insights with a practical tool for exploring cycle extremality in low-degree graphs.

Keywords: induced cycle, cubic graph, backtracking, pruning heuristics, computational study

Contents

Introduction	1
1 Terms and Terminology	2
2 Preliminaries	6
2.1 Induced Cycles	6
2.1.1 Problems and Complexity	7
2.1.2 Relation to Henning et al.	8
3 Upper Bound	9
3.1 General Upper Bound	9
3.2 Bridge-Decompositions	11
4 Induced Path Construction Method	13
4.1 Path-Growth Framework	13
4.2 Extension and Termination Operations	14
4.3 Path-Extending Closure	16
4.4 Avoiding Redundant Search	17
4.5 Implementation Outline	20
4.6 Baseline Comparison with Subset-Testing	23
5 Results	25
5.1 All Cubic Graphs by Connectivity and Girth	25
5.2 Classes with small induced cycles	26
5.2.1 (2,3,5)-graphs	31
5.2.2 (2,4,6)-graphs	32
5.3 Planar Cubic Graphs	33
5.4 Open Conjectures	34
Conclusion	36

List of Figures

1.1	A graph G with subgraphs G' and H : G' is an induced subgraph of G , but H is not.	3
1.2	A cycle C_6 (depicted in bold) with a chord ce , and induced cycles C_5 (on $abcef$) and C_3 (on cde).	4
2.1	A diamond-necklace N_8	7
3.1	A bridge-vertex b whose two non-bridge neighbours a, c are adjacent. Such b can never lie on an induced cycle.	11
4.1	Vertices b and c lie in $\beta_G(P)$, they are the neighbours that were banned when extending the path at x_1 and x_2 , respectively. The red line is the induced path after adding a	14
4.2	Running times of our induced-path backtracking algorithm (blue) versus the early-exit BA (orange), plotted against the number of vertices n and logarithmic time of computation.	24
5.1	Minimum and maximum values of the longest induced cycles $\iota(G)$ across each (n, κ, g) subclass.	26
5.2	A building block B_A contributing 2 vertices to a cycle coming from the left side. The two semi-edge pairs of B_A are both (x_1^*, x_2^*) , the first depicted in red facing left, the other one depicted in black facing right.	27
5.3	Two views of the building block B_B : The first one on the left with its own induced 4-cycle highlighted in red, and the second one on the right with the three vertices contributing to an external cycle lying on the green path.	28
5.4	Two views of the block B_C : The first one on the left with its own induced 5-cycle highlighted in red, and the second one on the right with the 4 vertices contributing to an external cycle lying on the green path.	28
5.5	Two views of the block B_D : The first one on the left with its internal induced 6-cycle highlighted in red, and the second one on the right with the 4 vertices contributing to an external cycle lying on the green path.	28

5.6	Two views of the block B_E : The first one on the left with one of its triangles highlighted in red representing the longest induced cycle within B_E , and the second one on the right with the 3 vertices contributing to an external cycle lying on the green path, here the path can also go through x_1 instead of x_0	29
5.7	Two views of the block B_F : The first one on the left with its internal induced 6-cycle, and the second one on the right with the 2 vertices contributing to an external cycle lying on the green path.	29
5.8	Fusion of two building blocks B', B'' . The 4 semi-edges $x_1^*, x_2^*, x_3^*, x_4^*$ depicted in green, connect to make 2 edges x_1x_3, x_2x_4	30
5.9	Join of 3 building blocks $B^{(A)}, B^{(B)}, B^{(C)}$. The 6 semi-edges $a_0^*, a_1^*, b_0^*, b_1^*, c_0^*, c_1^*$ depicted in green, connect to make 3 edges a_0c_0, c_1b_0, b_1a_1	30
5.10	Minimum and maximum values of the longest induced cycles $\iota(G)$ across each (n, g) subclass of planar graphs.	34

Introduction

The study of induced cycles in graphs combines insights from structural graph theory and algorithmic design. In general graphs, the problem of finding the largest induced r -regular subgraph is NP-hard already for $r = 2$.

This thesis fills that gap by presenting a backtracking algorithm that grows chord-free paths while maintaining a β -set of forbidden vertices to prevent chord formation. Key pruning rules—forced extensions when only one safe choice exists, removal of vertices whose contributions are settled, and abandonment of branches that cannot exceed the current best cycle length—drastically reduce the search space. We prove correctness through extension and termination lemmas and demonstrate that our method finds optimal induced cycles in milliseconds on graphs up to 28 vertices, compared to minutes for naive subset enumeration. Experiments on all bridgeless cubic graphs up to 24 vertices, planar graphs up to 28 vertices, and comparisons with baseline methods confirm both accuracy and efficiency.

The thesis unfolds as follows. Chapter 1 establishes notation and fundamental concepts of induced cycles. Chapter 2 surveys related work on extremal and algorithmic results for induced subgraphs. Chapter 3 derives upper bounds on longest induced cycles in cubic graphs and examines extremal families. Chapter 4 details our induced path construction method: definitions, the β -set mechanism, pruning strategies, and recursive pseudocode. Chapter 5 presents computational results, focuses on 2 specific classes with small induced cycles, formulates conjectures based on observed patterns in connectivity, girth, and planarity, and tests our algorithm against subset-testing benchmarks.

Chapter 1

Terms and Terminology

This chapter introduces key terms and concepts used throughout this paper, following the logical sequence of definitions as presented in Diestel's *Graph Theory* [1], as it provides a clear and structured approach to understanding fundamental graph-theoretic concepts. The definitions provided here form the foundation for understanding the structural and theoretical discussions in later chapters.

A *graph* G consists of a set of *vertices* V and a set of *edges* E , where each edge is a subset of vertices of size 2, in another words $E \subseteq \binom{V}{2}$. It is given that $V \cap E = \emptyset$, meaning no element belongs to both sets. The graph is then represented as a pair $G = (V, E)$.

If a graph G has a *vertex set* V , we say that it is a graph on V and refer to this set as $V(G)$, while its *edge set* is written as $E(G)$, these references are used regardless of the actual set names. For example, even if a graph is given as $H = (W, F)$, we still write $V(H)$ rather than $W(H)$.

When discussing elements of a graph, we often simplify notation. Instead of saying that a vertex v belongs to $V(G)$, we may simply say that v belongs to G . The same applies to edges: instead of writing $e \in E(G)$, we may say $e \in G$.

The *order* of a graph is given by the number of its vertices, denoted as $|G|$. Similarly, the number of edges in a graph is written as $\|G\|$. Based on its order, a graph can be classified as *finite*, *infinite*, *countable*, and so on. In this thesis, all graphs will be finite.

If v is a vertex, e is an edge and $v \in e$, we say that v is *incident* to e , in this case e is said to be an edge at v . An edge is incident to exactly two vertices, which are referred to as its *endvertices* or *ends*, and we say that the edge *joins* its two ends. An edge $\{x, y\}$ is commonly written as xy (or equivalently yx). If $x \in X$ and $y \in Y$, then xy is called an X – Y edge. The set of all X – Y edges within a set E is denoted by $E(X, Y)$. For singleton sets, for simplicity, we write $E(x, Y)$ instead of $E(\{x\}, Y)$, and the same goes for $E(X, y)$. The set of all edges in E that are incident to a vertex v is written as $E(v)$.

In a graph G , two vertices x and y are said to be *adjacent* (neighbours) if there is an edge connecting them, that is $xy \in G$. Similarly, two distinct edges are *adjacent* if they share a common end. A graph G is called *complete* if every pair of its vertices is adjacent. A complete graph with n vertices is denoted by K_n . A specific example is K_3 , which is referred to as a *triangle*.

Vertices or edges that are not adjacent to each other are referred to as *independent*. A set is called *independent* when no two of its elements are adjacent. This applies to both sets of vertices and sets of edges.

We define the union of two graphs G and G' as $G \cup G' := (V \cup V', E \cup E')$ and their intersection as $G \cap G' := (V \cap V', E \cap E')$. If these two graphs have no common vertices or edges, meaning $G \cap G' = \emptyset$, they are called *disjoint*.

A graph G' is said to be a *subgraph* of G if its vertex and edge sets are subsets of those of G , I. e., if $V' \subseteq V$ and $E' \subseteq E$. In this case, G is referred to as a *supergraph* of G' , which we write as $G' \subseteq G$. Informally, we may also say that G *contains* G' . If G' is a subgraph of G but not equal to it, then G' is called a *proper subgraph* of G .

G' is an *induced subgraph* of G if it includes all edges $xy \in E$ where both x and y are in V' , we denote it as $G' := G[V']$. This means that for any vertex set $U \subseteq V$, the notation $G[U]$ represents the subgraph formed by U along with all edges from G that have both ends in U .

Additionally, if H is any subgraph of G , not necessarily induced, we use the shorthand notation $G[H]$ instead of explicitly writing $G[V(H)]$.

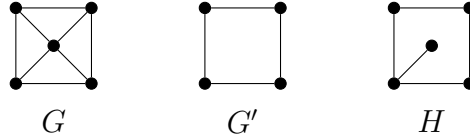


Figure 1.1: A graph G with subgraphs G' and H : G' is an induced subgraph of G , but H is not.

When U is a given set of vertices (usually from G), we write $G - U$ instead of $G[V \setminus U]$, representing the graph obtained after removing all vertices in $U \cap V$ along with their incident edges.

If U is a single vertex, meaning $U = \{v\}$, we simplify the notation to $G - v$ rather than $G - \{v\}$. Likewise, when F is a subset of $\binom{V(G)}{2}$, we use $G - F := (V, E \setminus F)$ and $G + F := (V, E \cup F)$. Following the same pattern as above, we abbreviate $G - \{e\}$ and $G + \{e\}$ as $G - e$ and $G + e$, respectively.

Let $G = (V, E)$ be a non-empty graph. For a vertex v in G , the set of its neighbours is denoted by $N_G(v)$, or simply $N(v)$. More generally, if U is a subset of V , the set of vertices in $V \setminus U$ that are adjacent to at least one vertex in U is called the *neighbours of U* and is denoted by $N(U)$.

The *degree* (or *valency*) of a vertex v , denoted as $d(v)$, represents the number of edges incident to v which is equal to the number of its neighbours, that is $d(v) = N(v)$. A vertex with $d(v) = 0$ is called *isolated*. A graph G has both a *minimum* and a *maximum degree*, denoted respectively as

$$\delta(G) := \min\{d(v) \mid v \in V\}, \quad \Delta(G) := \max\{d(v) \mid v \in V\}.$$

If every vertex in G has the same degree k , the graph is called *k-regular*, or just *regular*. When $k = 3$, the graph is referred to as *cubic*.

A non-empty graph $P = (V, E)$ is called a *path* if it has the form

$$V = \{x_0, x_1, \dots, x_k\}, \quad E = \{x_0x_1, x_1x_2, \dots, x_{k-1}x_k\},$$

where all vertices x_i for $i = 0, 1, \dots, n$ are distinct. The first vertex x_0 and the last vertex x_k are connected by P and are called its *ends*, while all other vertices $x_1, \dots, x_{k-1}$ are referred to as the *inner vertices* of P .

The *length* of a path is defined by the number of its edges. A path of length k is denoted as P_k . In the special case where $k = 0$, the path consists of a single vertex and is written as $P_0 = K_1$. When a path is given by an ordered sequence of vertices $x_0, x_1, \dots, x_k$, we write it as $P = x_0x_1 \dots x_k$ and refer to it as a path from x_0 to x_k (or between x_0 and x_k).

If $k \geq 3$ and $P = x_0 \dots x_{k-1}$ is a path, then adding the edge $x_{k-1}x_0$ creates a new graph $C := P + x_{k-1}x_0$, which is called a *cycle*. Similar to paths, the *length* of a cycle is defined by its number of edges, which in this case is also equal to the number of its vertices. A cycle of length k , with vertices $x_0, x_1, \dots, x_{k-1}$, can be written as $x_0 \dots x_{k-1}x_0$. Such a cycle is called a *k-cycle* and is denoted by C_k .

The *girth* of a graph G , denoted as $g(G)$, is the *minimum length* of a cycle contained in G , while the *circumference* of G is the *maximum length* of a cycle in G . If G has no cycles, the girth is set to ∞ , and the circumference is set to 0. A *chord* is an edge that joins two vertices of a cycle but is not itself part of the cycle. Therefore, an *induced cycle* in G is a cycle that forms an induced subgraph, meaning it has no chords.

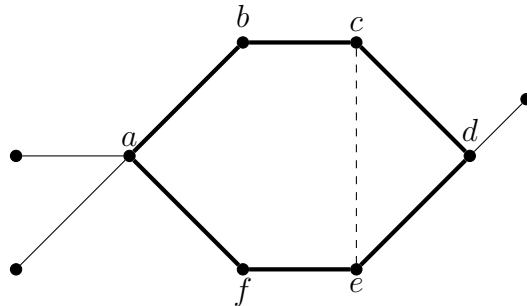


Figure 1.2: A cycle C_6 (depicted in bold) with a chord ce , and induced cycles C_5 (on $abcef$) and C_3 (on cde).

We write the *induced-cycle number* of G , denoted $\iota(G)$, for the length of a longest induced cycle in G . Formally,

$$\iota(G) = \max\{|C| : C \text{ is an induced cycle in } G\}.$$

A graph G is either *connected* or *disconnected*. It is called *connected* if there is a path between every pair of vertices in G . If G is not connected, we say it is *disconnected*. Let U be a subset of vertices from G . If the subgraph $G[U]$ is connected, then we also say that U itself is *connected in* G . A *component* of G is a maximal connected subgraph. An edge $e \in E(G)$ is a *bridge* (or *cut-edge*) if its removal increases the number of components of G . Equivalently, e lies on no cycle of G . When e is a bridge, writing

$$G - e = G_1 \cup G_2,$$

the two components G_1, G_2 are the *bridge-decomposition* of G along e .

A graph G is said to be *k-connected* for $k \in \mathbb{N}$ if it has more than k vertices and remains connected whenever fewer than k vertices are removed. In other words, at least k vertices must be deleted to disconnect the graph or separate two of its vertices. By definition, every non-empty graph is *0-connected*, while *1-connected* graphs are precisely those that are connected and contain more than one vertex. The largest integer k for which G remains k -connected is called the *connectivity* of G , denoted by $\kappa(G)$. This means that $\kappa(G) = 0$ if and only if G is either disconnected or consists of a single vertex (K_1). For complete graphs, the connectivity follows $\kappa(K_n) = n - 1$ for all $n \geq 1$.

If $|G| > 1$, then G is called *λ -edge-connected* if removing fewer than λ edges does not disconnect it. The largest integer λ for which this holds is known as the *edge-connectivity* of G , denoted by $\lambda(G)$. In particular, if G is disconnected, then its edge-connectivity is $\lambda(G) = 0$.

A classical result for regular graphs of degree three (cubic graphs) is that vertex-connectivity and edge-connectivity coincide:

$$\kappa(G) = \lambda(G).$$

Accordingly, in this thesis we shall simply refer to “connectivity” without distinguishing between the vertex and edge versions.

Chapter 2

Preliminaries

In this chapter, we look at the paper *Induced Cycles in Graphs* by Henning et al.[3]. This work provides valuable insights into the study of induced cycles, focusing on their properties and lower bounds within specific graph classes. By examining their results and methodologies, we aim to establish a foundation for understanding the current state of research and situate our work within this context.

2.1 Induced Cycles

Understanding the behavior of induced cycles in graphs begins with identifying general lower bounds on their size. A natural question arises: how large can the induced cycle number be for a graph, particularly when the graph structure imposes certain constraints? Henning et al.[3] address this question by introducing the concept of the induced cycle number, $c_{\text{ind}}(G)$, which measures the maximum size of an induced 2-regular subgraph in a graph G

$$c_{\text{ind}}(G) = \max\{|H| : H \subseteq G \text{ is an induced 2-regular subgraph}\}.$$

Note that a 2-regular subgraph can be a disjoint union of cycles, therefore, it doesn't have to be connected. This concept serves as a fundamental tool for studying cycles in graphs and provides a starting point for understanding the limits of induced cycles under various structural assumptions.

By focusing on cubic graphs, the authors established a result that sets the baseline for further exploration:

Theorem 1. [3] *If G is a cubic graph of order n , then $c_{\text{ind}}(G) > n/4$.*

While this lower bound applies broadly to all cubic graphs, its generality also limits its strength. It does not take into account additional structural properties that might enable longer induced cycles. This observation naturally leads to the question of

whether stronger results can be obtained by imposing further restrictions on the graph structure.

To investigate this question, the authors focused on *claw-free* cubic graphs. Claw-free graphs, which exclude the $K_{1,3}$ induced subgraph, impose tighter conditions on the graph structure, and these restrictions allow for a significant improvement in the lower bound for the induced cycle number. This result is particularly relevant for our work, as we aim to explore how different graph classes influence the induced cycle properties.

Theorem 2. [3] *If G is a cubic claw-free graph of order n , then $c_{\text{ind}}(G) > 13n/20$.*

Here we can see the importance of claw-free properties in achieving longer induced cycles. Compared to Theorem 1, this result highlights how additional constraints on the graph structure can substantially increase the induced cycle number. For our study, which focuses on different subclasses of cubic graphs, Theorem 2 serves as a motivation and benchmark for exploring further improvements.

To demonstrate how close these bounds can get to optimal, the authors constructed a family of cubic graphs that achieves a relatively high value for $c_{\text{ind}}(G)$. One such example is the diamond-necklace graph:

Observation 1. [3] *If $G \in \mathcal{N}_{\text{cubic}}$ is a diamond-necklace of order n , then $c_{\text{ind}}(G) = 3n/4$.*

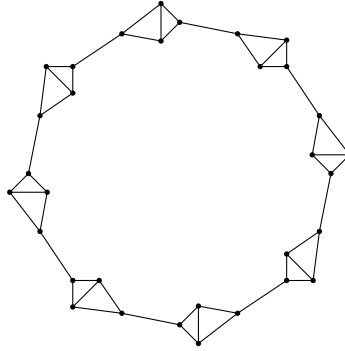


Figure 2.1: A diamond-necklace N_8 .

Observation 1 is particularly relevant because we will later demonstrate that this value serves as the upper bound for our parameter $\iota(G)$. The diamond-necklace graph thus acts as both a guiding example and a benchmark for what can be achieved in cubic graphs with specific structural properties.

2.1.1 Problems and Complexity

An important challenge addressed in this work concerns the computational complexity of determining the induced cycle number $c_{\text{ind}}(G)$. Understanding whether this problem

is computationally difficult helps to contextualise the practical challenges of finding long induced cycles, even in structured graph classes.

INDUCED CYCLES PROBLEM:

Instance: A given graph G and a positive integer k .

Question: Is $c_{\text{ind}}(G) \geq k$?

The authors conjecture that this problem remains NP-complete for cubic graphs:

Conjecture 1. [3] *INDUCED CYCLES is NP-complete for cubic graphs.*

The NP-completeness conjecture emphasizes the inherent difficulty of the problem and highlights why studying subclasses of cubic graphs, where results may be more tractable, is both necessary and valuable. While this thesis does not address the complexity question directly, the conjecture underscores the importance of developing techniques for analyzing induced cycles efficiently.

2.1.2 Relation to Henning et al.

Henning et al. [3] address the broad challenge of finding large induced 2-regular subgraphs—possibly a union of disjoint cycles—and establish general lower bounds on their size via the parameter

$$c_{\text{ind}}(G) = \max\{|H| : H \subseteq G \text{ is an induced 2-regular subgraph}\}.$$

By contrast, we concentrate on the *longest single induced cycle* in a cubic graph, i.e. the connected 2-regular subgraph of maximum order. Since every induced cycle is a 2-regular subgraph, our problem can be seen as a refinement of theirs, focusing on one component rather than a union. This narrowing of scope lets us design specialized algorithms and prove sharper bounds for

$$\iota(G) = \max\{|C| : C \subseteq G \text{ is an induced cycle}\}.$$

Chapter 3

Upper Bound

In this chapter, we establish several upper bounds for the longest induced cycle in cubic graphs. First, we examine the general upper bound, building on Observation 1 to show that it applies beyond the diamond-necklace construction given by Henning et al. [3]. We then present a subclass of cubic graphs where this general bound can be overly optimistic, as no induced cycle in this subclass can ever reach the general upper bound.

3.1 General Upper Bound

Henning et al. [3] provided a construction (the *diamond-necklace* graph, recalled in Figure 2.1), showing that for certain cubic graphs G of order n , one can achieve an induced 2-regular subgraph (or set of induced cycles) of order $\frac{3n}{4}$. Here, we show that this is not only an example-specific bound but indeed the *global* upper limit on the length of any induced cycle in a cubic graph.

Lemma 1. *Let G be a cubic graph on n vertices. Then any induced cycle $C \subseteq G$ satisfies*

$$|C| \leq \frac{3n}{4}.$$

Proof. Assume, for contradiction, that there is an induced cycle $C \subseteq G$ with

$$|C| > \frac{3n}{4}.$$

Because G is cubic, each vertex $v \in C$ has degree 3 in G . Two of these edges connect v to its neighbours on C , and since C is induced, v has *exactly one* neighbour outside of C . Denote by N_C the set of all such neighbours of vertices in C lying outside of C . Formally,

$$N_C = \{w \in V(G) \setminus C \mid \exists v \in C : vw \in E(G)\}.$$

Since C and N_C are disjoint sets, we must have

$$|C| + |N_C| \leq n \implies |N_C| \leq n - |C|.$$

Given our assumption $|C| > \frac{3n}{4}$, it follows that

$$|N_C| \leq n - |C| < n - \frac{3n}{4} = \frac{n}{4}.$$

Hence,

$$|N_C| < \frac{n}{4}.$$

Each $v \in C$ contributes exactly *one* neighbour to N_C , so for the edge cut E between C and N_C is true that $|E| \leq |C|$. However, a single vertex in N_C can be the outside neighbour of *at most* 3 vertices in C , since G is cubic. Thus, in the most efficient packing, one vertex of N_C might serve as an external neighbour for up to three vertices in C , therefore $\frac{|C|}{3} \leq |E|$. Consequently,

$$|N_C| \geq \frac{|C|}{3}.$$

By our assumption $|C| > \frac{3n}{4}$, we get

$$\frac{|C|}{3} > \frac{3n}{4} \cdot \frac{1}{3} = \frac{n}{4}.$$

Therefore,

$$|N_C| > \frac{n}{4}.$$

The two inequalities derived contradict each other:

$$|N_C| > \frac{n}{4} \quad \text{and} \quad |N_C| < \frac{n}{4}.$$

Hence, our assumption that $|C| > \frac{3n}{4}$ must be false. We conclude that no induced cycle in a cubic graph can exceed $\frac{3n}{4}$ vertices. $\square$

Corollary 1. *If G is a cubic graph on n vertices, then*

$$\max\{|C| : C \text{ is an induced cycle in } G\} \leq \frac{3n}{4}.$$

While it is true that most cubic graphs admit induced cycles approaching $\frac{3n}{4}$ in length, there are simple—but non-trivial—families whose longest induced cycles can never reach it. One such family is introduced in the next section.

3.2 Bridge-Decompositions

We now present a family whose induced cycles can never aspire to acquire the length of the general upper bound. Every graph with at least one bridge belongs to this family. Recall that a bridge in a graph is an edge whose removal increases the number of connected components. Necessarily, any bridge cannot lie on a cycle—removing it would disconnect the two ends of that cycle—so every induced cycle must lie entirely within one of the bridge-free components. Consequently, if G is a cubic graph and e is a bridge, writing $G - e = G_1 \cup G_2$ for its two components, we have

$$\iota(G) = \max\{\iota(G_1), \iota(G_2)\}.$$

More generally, suppose that removing all bridges from G partitions it into k components $G_1, \dots, G_k$, where $|V(G_i)| = v_i$ for each i and $\sum_{i=1}^k v_i = n$.

Every one of these components becomes subcubic as only the bridge vertices now have degree smaller than 3 and the rest stay with degree 3 as before. In later chapters, when we present our full algorithm for finding the longest induced cycles in cubic graphs, we will also show how it could be applied in this case.

Trying to determine the upper bound in this class, we first get a limit given by the number of vertices in each component. Looking at the general split mentioned earlier, each component G_i can theoretically use every vertex to form an induced cycle, making v_i the upper bound for G_i . That would mean that

$$\iota(G) \leq \max(v_i).$$

for $i = 1, \dots, k$ where k is the number of connected components of G after removing all bridges.

We could make this tighter by identifying how many of the bridge vertices in a given component have their non-bridge neighbours adjacent to one another. In such a case (see Figure 3.1), any cycle passing through the bridge-vertex b would also have to visit both of its neighbours a and c , but since a and c are themselves joined, that would create a chord—hence b cannot lie on any induced cycle.

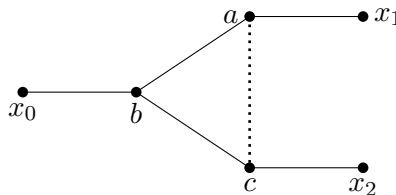


Figure 3.1: A bridge-vertex b whose two non-bridge neighbours a, c are adjacent. Such b can never lie on an induced cycle.

However, we won't be addressing this issue in more detail, as this knowledge alone is enough to discourage us from testing on graphs with bridges. Accordingly, throughout the remainder of this thesis, we will restrict all computational experiments and algorithmic developments to *bridgeless* cubic graphs.

Chapter 4

Induced Path Construction Method

We present a backtracking algorithm for finding longest induced cycles in cubic graphs. Starting from a seed edge or vertex, the method grows chord-free paths while tracking a banned-vertex set (the β -set) to avoid chords. Whenever a path can close into an induced cycle, we update the best length and continue extending.

Key components include formal definitions of induced paths, extensions, and terminations, plus the β -set. To manage larger graphs, we employ two pruning strategies: the residual check, which abandons branches unable to exceed the current best cycle, and the peeling reduction, which removes already-settled vertices to shrink the search space.

We then outline the recursive pseudocode and prove correctness via extension and termination lemmas. By leveraging cubic-degree limits, deterministic extensions, and global futility pruning, this approach finds longest induced cycles in milliseconds on instances where brute-force would take minutes.

Finally, we compare our method to a naïve subset-testing benchmark algorithm (BA)—both exhaustive and early-exit variants—demonstrating that, while the BA requires minutes on $n = 28$ instances, our algorithm completes in milliseconds, yet consistently agrees on the true value of $\iota(G)$. Together, these results validate both the correctness and the efficiency of the induced-path approach across diverse cubic graphs.

4.1 Path-Growth Framework

Recall that an induced subgraph $G' = G[U]$ on $U \subseteq V(G)$ contains every edge of G whose ends both lie in U . An *induced path*

$$P = x_0x_1 \cdots x_k$$

is an induced subgraph which itself is a path from x_0 to x_k .

To control chord-creation when we extend P , we define a *path-banned set* $\beta_G(P)$. Formally we set

$$\beta_G(P) = \bigcup_{i=1}^{k-1} (N_G(x_i) \setminus V(P)).$$

Here, the subscript G reminds us that neighbourhoods are taken in the host graph, and the argument P indicates which vertices are part of the path P so far.

We say P is *extendable* if each end-vertex has at least one neighbour lying outside

$$V(P) \cup \beta_G(P),$$

and *deterministically extendable* if at least one end-vertex has exactly one such neighbour. Figure 4.1 shows the deterministic case: one neighbour of x_3 lies in $\beta_G(P)$, so only the other may extend.

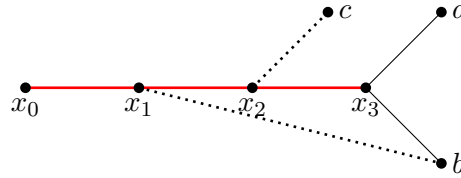


Figure 4.1: Vertices b and c lie in $\beta_G(P)$, they are the neighbours that were banned when extending the path at x_1 and x_2 , respectively. The red line is the induced path after adding a .

Finally, P is *closable* if there is an edge in G joining its two end-vertices that does not already lie in P , for then adding that edge closes P into an induced cycle.

4.2 Extension and Termination Operations

We first describe how to extend an arbitrary path P by one vertex, and then how to terminate (close) a path into a cycle.

An *path extension* of a path

$$P = x_0 x_1 \cdots x_k$$

at one of its end-vertices is obtained by picking that end-vertex, say x_k , and any neighbour

$$w \in N_G(x_k) \setminus V(P)$$

and forming the new, longer path

$$\text{ex}(P, x_k, w) = x_0 x_1 \cdots x_k w.$$

Dually, to extend at the other end x_0 , one picks

$$w \in N_G(x_0) \setminus V(P)$$

and sets

$$\text{ex}(P, x_0, w) = w x_0 x_1 \cdots x_k.$$

Lemma 2. *Let $P = x_0 x_1 \cdots x_k$ be an induced path in a cubic graph G , and let $w \in N_G(x_k) \setminus V(P)$. Then $\text{ex}(P, x_k, w)$ is an induced path if and only if w does not lie in the banned set $\beta_G(P)$.*

Proof. We show that both implications are true, thus proving the equivalence.

($\Rightarrow$) If $\text{ex}(P, x_k, w)$ is an induced path, then $w \notin \beta_G(P)$.

If $w \in \beta_G(P)$, then by definition there exists some interior x_i ($1 \leq i \leq k-1$) with

$$w \in N_G(x_i) \setminus V(P).$$

But then the edge $w x_i$ would be a chord in $\text{ex}(P, x_k, w)$, contradicting that $\text{ex}(P, x_k, w)$ is induced.

($\Leftarrow$) If $w \notin \beta_G(P)$, then $\text{ex}(P, x_k, w)$ is an induced path.

Since $w \notin V(P)$, appending w does not repeat any vertex. And because w does not lie in any $N_G(x_i) \setminus V(P)$ for $1 \leq i \leq k-1$, there is no edge $w x_i$ creating a chord. Hence $\text{ex}(P, x_k, w)$ is chord-free and so an induced path. □

Now we introduce the *path termination* operation. Suppose

$$P = u x_1 x_2 \cdots x_{k-1} v$$

is a path whose end-vertices u and v happen to be adjacent in G by an edge $e = \{u, v\}$ not already used in P . We then define its termination to be the cycle

$$t(P) = u x_1 x_2 \cdots x_{k-1} v u.$$

Lemma 3. *If P is a closable induced path in G , then $t(P)$ is an induced cycle.*

Proof. Write $C = t(P)$. By definition, we simply added the edge $\{u, v\}$ between the two distinct end-vertices of the induced path P . No vertex is repeated, and no new chord can appear—any chord would join two vertices of P , but all such potential chords were forbidden by the banned-set mechanism when P was built. Hence C is chord-free and so is an induced cycle. □

4.3 Path-Extending Closure

Given an initial induced path $P = uv$, we define its *path-extending closure* $\mathcal{C}_G(P)$ as the maximal family of induced cycles in G containing P that one can obtain by alternately applying extension and termination. When extending a path Q at one of its end-vertices x , Lemma 2 guarantees that the induced property is preserved by choosing a neighbour

$$w \in N_G(x) \setminus (V(Q) \cup \beta_G(Q)).$$

Whenever a path becomes closable—its two ends share an edge not already in the path—we can apply the termination step $t(Q)$ to close it into an induced cycle. By insisting on every extension on $w \notin \beta_G(Q)$, and only terminating when a chord-closing edge exists, we ensure every member of $\mathcal{C}_G(P)$ is chord-free. Maximality follows because no further extension or termination can be applied to any cycle in $\mathcal{C}_G(P)$.

Recall that $\iota(G)$ denotes the length of the longest induced cycle in G :

$$\iota(G) = \max\{|C| : C \text{ is an induced cycle in } G\}.$$

We can refine this notion to obtain metrics on the longest induced cycle in a graph containing a prescribed object. For any edge $e \in E(G)$, let

$$\iota_e(G) = \max\{|C| : C \text{ is an induced cycle in } G \text{ containing } e\}.$$

Equivalently, since every such cycle appears in the closure of some initial path using e ,

$$\iota_e(G) = \max\{|C| : C \in \mathcal{C}_G(P) \text{ for some initial path } P \ni e\}.$$

Dually, for any vertex $v \in V(G)$, define

$$\iota_v(G) = \max\{|C| : C \text{ is an induced cycle passing through } v\}$$

also as

$$\iota_v(G) = \max\{|C| : C \in \mathcal{C}_G(P) \text{ for some initial path } P \text{ with } v \in V(P)\}.$$

Clearly,

$$\iota(G) = \max_{e \in E(G)} \iota_e(G) = \max_{v \in V(G)} \iota_v(G).$$

We can also see, that for a vertex $v \in V(G)$ and its neighbours $u \in N_G(v)$

$$\iota_v(G) = \max\{\iota_e(G) : \text{where } e = vu\},$$

4.4 Avoiding Redundant Search

All of our machinery—induced paths, extensions, terminations, banned sets, closures, and the parameters ι , ι_v , ι_e —remain valid in any subcubic graph (every vertex of degree at most 3). We now fix a *futile-vertex set* $\Phi \subseteq V(G)$ to record those vertices v for which we have already determined $\iota_v(G)$, i.e. the length of the longest induced cycles passing through v .

Once we have computed every path-extending closure for all initial paths beginning in Φ , no induced cycle of maximum length can avoid Φ . Equivalently, any cycle of length $\iota(G)$ either uses some vertex in Φ or lies entirely in the smaller graph $G - \Phi$. It follows that we may safely “peel off” Φ and restrict further search to $G - \Phi$.

This idea yields the following reduction lemma.

Lemma 4. *Let G be a subcubic graph and let $v \in V(G)$. Then*

$$\iota(G - \Phi) = \max\{\iota_v(G - \Phi), \iota(G - (\Phi \cup \{v\}))\}.$$

Proof. Write $\Phi' = \Phi \cup \{v\}$. Let C be any longest induced cycle in G . There are two cases:

Case 1: $v \in V(C)$. Then C lies in the subgraph $G - \Phi$, and by definition of ι_v we have

$$|C| \leq \iota_v(G - \Phi).$$

Case 2: $v \notin V(C)$. Then C is entirely contained in $G - \Phi'$, so

$$|C| \leq \iota(G - \Phi').$$

In either case,

$$|C| \leq \max\{\iota_v(G - \Phi), \iota(G - \Phi')\}.$$

Since C was chosen to have length $\iota(G)$, this shows $|C| = \iota(G) \leq \max\{\iota_v(G - \Phi), \iota(G - \Phi')\}$.

In contrast, both $\iota_v(G - \Phi)$ and $\iota(G - \Phi')$ are clearly bounded above by $\iota(G)$, because any induced cycle in either $G - \Phi$ or $G - \Phi'$ is also an induced cycle in G . Hence

$$\iota(G) \geq \max\{\iota_v(G - \Phi), \iota(G - \Phi')\}.$$

Putting it all together, we get

$$\max\{\iota_v(G - \Phi), \iota(G - \Phi')\} \geq \iota(G) \geq \max\{\iota_v(G - \Phi), \iota(G - \Phi')\}.$$

□

This lemma yields a simple recursion: to compute $\iota(G - \Phi)$ one either extends through v (paying cost $\iota_v(G - \Phi)$) or else “peels off” v and recurses on $G - (\Phi \cup \{v\})$. In other words, at each step we branch on whether any longest induced cycle uses v or not, giving a clean divide-and-conquer structure.

Lemma 4 not only lets us peel off one vertex at a time, but by iterating this choice across a fixed vertex ordering to shows that we need only examine some initial segment before guaranteeing we have found the global maximum. More precisely:

Lemma 5. *Let G be a subcubic graph on n vertices, and fix an integer $n_0 \leq n$. Order the vertices of G as $v_1, \dots, v_{n_0}, v_{n_0+1}, \dots, v_n$. Let*

$$\ell = \max\{\iota_{v_1}(G), \iota_{v_2}(G - \{v_1\}), \dots, \iota_{v_{n_0}}(G - \{v_1, \dots, v_{n_0-1}\})\}.$$

If $\ell \geq n - n_0$, then in fact $\ell = \iota(G)$.

Proof. For $1 \leq i \leq n_0$, write

$$G_i = G - \{v_1, v_2, \dots, v_{i-1}\},$$

so that $G_1 = G$ and $|V(G_i)| = n - i + 1$. By definition,

$$\ell = \max_{1 \leq i \leq n_0} \iota_{v_i}(G_i),$$

so ℓ is the maximal length of some induced cycle already found.

Now, any induced cycle contained entirely in G_{n_0+1} can have at most $|V(G_{n_0+1})| = n - n_0$ vertices. Since $\ell \geq n - n_0$, no cycle in G_{n_0+1} can exceed length ℓ . On the other hand, Lemma 4 tells us that every longest induced cycle in G either passes through one of $v_1, \dots, v_{n_0}$ (and so was counted in one of the $\iota_{v_i}(G_i)$) or lies entirely in G_{n_0+1} . In either case, its length is at most ℓ . Hence

$$\iota(G) = \ell,$$

□

In practice, Lemma 5 implies that as soon as our running maximum

$$\ell = \max_{1 \leq i \leq n_0} \iota_{v_i}(G - \{v_1, \dots, v_{i-1}\})$$

reaches $|V(G)| - n_0$, we can stop processing further vertices—no matter how many remain in $G - \Phi$, none can lie on an induced cycle longer than ℓ . Consequently, it is unnecessary to initiate path-extending closures from every vertex; instead, one proceeds down the chosen vertex ordering only until the current best length ℓ exceeds the number of vertices left in the graph.

Another pruning heuristic exploits the banned-set $\beta_G(P)$ itself. Recall that $\beta_G(P)$ records exactly those vertices whose inclusion would create a chord if appended to P .

Another way to prune is to look at the *banned set* itself. Recall that for any induced path

$$P = x_0x_1 \cdots x_{x-1},$$

the set

$$\beta_G(P) = \bigcup_{i=1}^{x-1} (N_G(x_i) \setminus V(P))$$

consists of exactly those vertices whose addition would create a chord. Hence, once P has length x , the only vertices available to extend it lie in

$$V(G) \setminus (V(P) \cup \beta_G(P)).$$

If—even using *all* of these remaining vertices—we cannot build a cycle longer than our current record L , there is no point in exploring further extensions of P . Formally:

Lemma 6. *Let G be a subcubic graph and let*

$$P = x_0x_1 \cdots x_{x-1}$$

be an induced path of length $|P| = x$. Suppose that during our search we have already found an induced cycle of length L . If

$$|V(G) \setminus (V(P) \cup \beta_G(P))| \leq L - x,$$

then no extension of P can close to an induced cycle longer than L . Hence all further extensions of P may be abandoned.

Proof. Any induced cycle obtained by extending P must use only vertices in

$$V(P) \cup (V(G) \setminus (V(P) \cup \beta_G(P))),$$

namely the current path plus the “safe” vertices outside its banned set. There are

$$x + |V(G) \setminus (V(P) \cup \beta_G(P))| \leq x + (L - x) = L$$

vertices available in total. Thus any cycle formed from P can have size at most L , so cannot improve our record. We may therefore prune this branch of the search. $\square$

Together, the two pruning rules—peeling off futile vertices (Lemmas 4–5) and banning chord-creating extensions (Lemma 6)—dramatically reduce the search space. In the next section, we show how these ideas come together in a concrete implementation.

4.5 Implementation Outline

We now present the pseudocode that brings together the path-growth framework, extension and termination lemmas, and pruning strategies from this chapter into a single, coherent recursive routine for computing $\iota(G)$.

Require: A cubic graph $G = (V, E)$

Ensure: $\iota(G)$, the length of some longest induced cycle in G

```

1:  $\Phi \leftarrow \emptyset$  ▷ futile-vertex set
2:  $\ell \leftarrow 0$  ▷ global best cycle length
3: function EXTENDSEARCH( $P, \beta, \ell_P$ ) ▷ try all safe extensions of path  $P$ ,  
updating local best  $\ell_P$ 
4:   if  $|V(G)| - (|V(P)| + |\beta_G(P)|) \leq \ell_P$  then ▷ by Lemma6
5:     return
6:   end if
7:   if  $\exists$  end-vertex  $x$  with exactly one safe neighbour  $w$  then ▷ deterministic  
extension
8:     PROCESSEXTENSION( $P, \beta, \ell_P, x, w$ )
9:     return
10:  end if
11:  for all end-vertices  $x$  and all safe neighbours  $w \in N_G(x) \setminus (V(P) \cup \beta)$  do
12:    PROCESSEXTENSION( $P, \beta, \ell_P, x, w$ )
13:  end for
14: end function
15: function PROCESSEXTENSION( $P, \beta, \ell_P, x, w$ )
16:    $P' \leftarrow \text{ex}(P, x, w)$ 
17:    $\beta' \leftarrow \beta \cup (N_G(x) \setminus \{\text{prev}(x), w\})$ 
18:   if  $P'$  is closable then
19:      $C \leftarrow t(P')$ 
20:      $\ell_P \leftarrow \max(\ell_P, |C|)$ 
21:   end if
22:    $\ell_P \leftarrow \text{EXTENDSEARCH}(P', \beta', \ell_P)$ 
23:   return
24: end function
25: for all  $v \in V$  in some fixed ordering do
26:   if  $\ell \geq |V| - |\Phi|$  then
27:     break ▷ by Lemma 5
28:   end if
29:   for all  $u \in N_G(v)$  do
30:      $P \leftarrow [v, u]$ 

```



```

31:       $\beta \leftarrow \Phi \cup \beta_G(P)$ 
32:       $\ell_P \leftarrow 0$ 
33:       $\ell_P \leftarrow \text{EXTENDSEARCH}(P, \beta, \ell_P)$ 
34:       $\ell \leftarrow \max(\ell, \ell_P)$ 
35:  end for
36:   $\Phi \leftarrow \Phi \cup \{v\}$ 
37: end for
38: return  $\ell$ 

```

Explanation of the Pseudocode

The algorithm proceeds in four phases:

1. Outer Loop & Futility Check.

We iterate over all vertices $v \in V(G)$ in some fixed order, maintaining a “futile” set Φ of vertices whose induced-cycle lengths ι_v have already been determined. After processing each v , we add it to Φ . Before beginning work on v , we test

$$\ell \geq |V| - |\Phi|,$$

and if true, we break immediately by Lemma 5: no vertex outside Φ can lie on an induced cycle longer than our current best ℓ .

2. Edge-Seeded Recursive Search.

For each neighbour $u \in N_G(v)$ of the current seed-vertex v , we initialize the path $P = (v, u)$ and its banned set $\beta = \Phi \cup \beta_G(P)$, so we only search in $G - \Phi$. We then invoke

$$\text{EXTENDSEARCH}(P, \beta, \ell_P),$$

which returns the length ℓ_P of the longest induced cycle discoverable from that seed. After the recursive call completes, we update our global best $\ell \leftarrow \max(\ell, \ell_P)$.

3. The Core Recursion: EXTENDSEARCH.

This function explores *all* safe extensions of the current path P , keeping track of the local best cycle length ℓ_P , while also making sure we won’t do unnecessary searches:

- We first use Lemma 6 to check whether we are able to exceed the current longest cycle length by counting how many vertices we can visit.

- After that we check for a *deterministic extension*—an end-vertex x that has *exactly one* neighbour outside $V(P) \cup \beta$. If such a forced move exists, we perform it immediately (via `PROCESSEXTENSION`) and return, since no other choice can arise.
- Otherwise, for *each* end-vertex x and each safe neighbour $w \in N_G(x) \setminus (V(P) \cup \beta)$, we call `PROCESSEXTENSION`(P, β, ℓ_P, x, w).

The recursion naturally backtracks as soon as each call to `PROCESSEXTENSION` returns.

4. Single-Step Extension: `PROCESSEXTENSION`.

Given a path P , a banned set β , a current local best ℓ_P , an end-vertex x , and a choice w , this helper does exactly one extension plus possible closure:

1. Form the new path $P' = \text{ex}(P, x, w)$. By Lemma 2, choosing $w \notin \beta$ ensures P' remains induced.
2. Update the banned set to
$$\beta' = \beta \cup (N_G(x) \setminus \{\text{prev}(x), w\}).$$
3. If P' is closable, form the cycle $C = t(P')$ (Lemma 3) and set $\ell_P \leftarrow \max(\ell_P, |C|)$.
4. Recurse: $\ell_P \leftarrow \text{EXTENDSEARCH}(P', \beta', \ell_P)$.

After returning, control goes back to whichever branch of `EXTENDSEARCH` invoked it, naturally exploring all possibilities.

This recursive decomposition cleanly separates:

- *Futility and banned set pruning* (outer loop + break conditions),
- *Choice logic* (deterministic vs. nondeterministic extension), and
- *Extension + closure correctness* (via the two helper lemmas).

In practice, this structure yields efficient backtracking: cubic-degree constraints limit each step to at most two safe neighbours, deterministic extensions avoid branching when possible, and futility pruning halts the outer search as soon as no longer cycle can be formed.

Complexity

Although the core backtracking search remains exponential in the worst case—since each extension step may branch into up to two new paths—our additional constraints yield dramatic pruning. The cubic-degree bound limits each vertex to at most two extensions, the banned-set test prevents any chord-creating moves, and the futility break of Lemma 5 halts the outer loop early whenever the current best cycle length exceeds the size of the remaining graph. Combined, these features reduce the practical running time from minutes (for naive subset-enumeration at $n = 28$) to mere milliseconds on the same inputs.

4.6 Baseline Comparison with Subset-Testing

As a sanity-check on correctness, we used a very simple “benchmark algorithm” (BA) to verify that our induced-path backtracking method indeed finds some longest induced cycle in every test graph. For curiosity’s sake we also compare performance, although it is clear that BA is orders of magnitude slower.

BA comes in two variants, both of which work on the same collection of bridgeless cubic graphs of order $8 \leq n \leq 28$:

First, the *exhaustive* variant searches for

$$k = \lfloor \frac{3n}{4} \rfloor, \lfloor \frac{3n}{4} \rfloor - 1, \dots$$

and for each k it enumerates *all* vertex-subsets of size k , testing whether each subset induces a single cycle of length k . It collects every such maximum-length induced cycle before halting. In effect this serves as a ground-truth oracle for $\iota(G)$, but its running time grows explosively (on a single $n = 28$ graph it took about 8 minutes).

Second, the *early-exit* variant performs the same descending- k scan, but stops immediately upon finding the *first* induced cycle of length k . This still correctly returns $\iota(G)$, yet on $n = 28$ graphs its worst-case time falls to under 6 minutes—still vastly slower than our backtracking implementation, which finishes in milliseconds (the slowest single run on $n = 28$ was 3 ms).

In every test graph both BA variants agreed with our algorithm on the true value of $\iota(G)$. Interestingly, both take longer on graphs whose longest induced cycle is smaller: since they begin at the upper bound $\lfloor 3n/4 \rfloor$ and only decrement k when no cycle is found, a graph with $\iota(G) = 18$ on $n = 28$ can cost several hundred seconds before its first witness cycle appears (and the exhaustive run then required roughly 500 s), whereas graphs with $\iota(G) = 20$ cost only about 94 s.

Figure 4.2 illustrates the direct comparison of our algorithm against the early-exit BA. Although one could also plot runtime as a function of the true cycle length, the

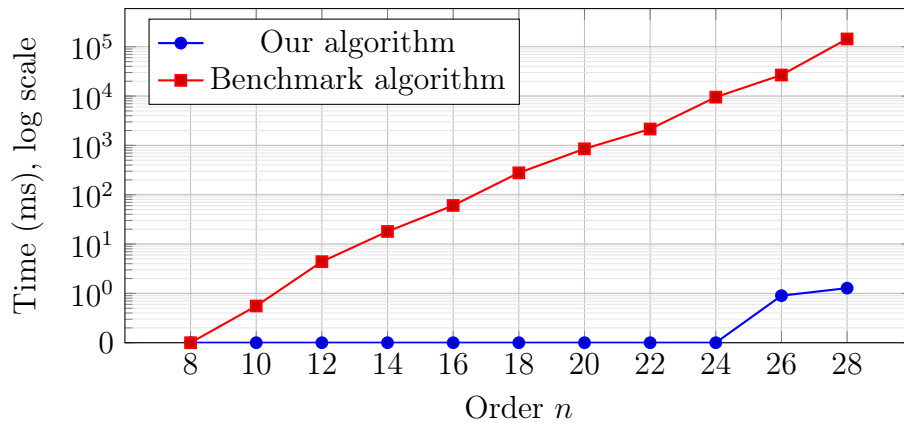


Figure 4.2: Running times of our induced-path backtracking algorithm (blue) versus the early-exit BA (orange), plotted against the number of vertices n and logarithmic time of computation.

dominant dependence on n together with our futility-pruning behavior make vertex-count the most informative horizontal axis here.

Chapter 5

Results

In this chapter we present an empirical evaluation of our induced-path backtracking algorithm on all bridgeless cubic graphs of order $8 \leq n \leq 28$. We begin by examining all bridgeless cubic graphs generated by **genreg**, classifying them by vertex-connectivity $\kappa \in \{2, 3\}$ and girth $g \in \{3, 4, 5, 6, 7\}$, and plotting the minimum and maximum longest induced-cycle lengths $\iota(G)$ in each subclass. We then specialize to planar cubic graphs produced by **plantri**, showing how planarity raises the lower bound on $\iota(G)$ across girth classes. Next, we state a series of conjectures distilled from these observations, formalizing patterns in girth- and connectivity-based minima and the persistence of the $\lfloor 3n/4 \rfloor$ upper bound.

5.1 All Cubic Graphs by Connectivity and Girth

We used **genreg**[4] to generate every bridgeless cubic graph on n vertices for $8 \leq n \leq 24$, we stopped at 24 because the time it would take to generate bigger graphs is too high. For each graph we recorded its vertex-connectivity $\kappa \in \{2, 3\}$ and its girth $g \in \{3, 4, 5, 6, 7\}$ and divided them into subclasses represented by these parameters (κ, g) . Within each of these subclasses we computed $\iota(G)$, the length of the longest induced cycles for every graph G .

Figure 5.1 displays, for each n and each *pair* (κ, g) , the minimum and maximum values of $\iota(G)$ observed across all graphs in that subclass.

A clear pattern emerges. In every (κ, g) subclass with $\kappa = 2$ that allows girth $g = 3$, the smallest observed $\iota(G)$ is always exactly 5. Similarly, when $\kappa = 2$ and $g = 4$, the minimal length $\iota(G)$ is only one longer, therefore 6. We paid special attention to those classes and will address them in the next section (5.2). Meanwhile, the largest observed $\iota(G)$ in each subclass almost always approaches the theoretical ceiling of $\lfloor \frac{3n}{4} \rfloor$ given by Lemma 1, growing nearly linearly in n .

Thus, even when we refine our data by both connectivity and girth, the phenomenon

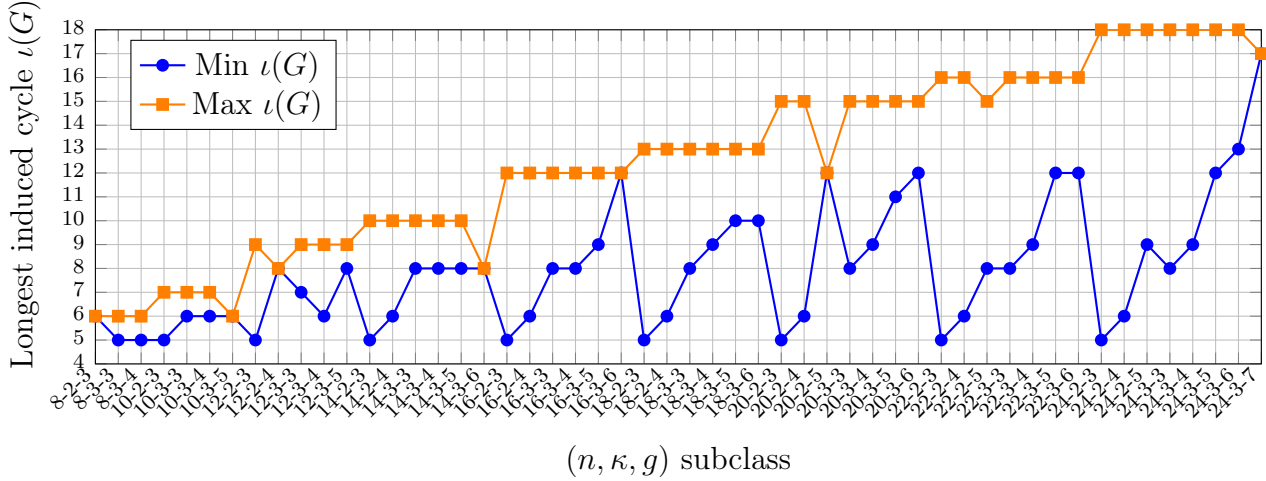


Figure 5.1: Minimum and maximum values of the longest induced cycles $\iota(G)$ across each (n, κ, g) subclass.

persists: only the connectivity 2 with girth 3 and 4 subclasses admit very small induced-cycles.

5.2 Classes with small induced cycles

While performing the evaluation in Section 5.1, we observed that in each $\kappa = 2$ subclass there exist graphs that attain the conjectured lower bound on ι : specifically, some graphs achieve $\iota = 5$ when $g = 3$ and $\iota = 6$ when $g = 4$. Motivated by these extreme examples, we focused on those two classes to uncover their underlying structure. By analyzing representative graphs, we identified a set of building blocks and gluing operations that appear to generate every member of the $(2, 3, 5)$ and $(2, 4, 6)$ families. We now introduce the foundational definitions and baselines needed for this decomposition.

First, an edge which is adjacent to only one vertex (has only one end) at one side and is empty on the other, is called *semi-edge*. When referring to a semi-edge incident with a vertex v , we will use the notation of v^* . When we connect two semi-edges v^* and u^* , we obtain an edge uv . If there are multiple semi-edges incident with a single vertex, we will differentiate otherwise. The set of semi-edges of a graph G is denoted as $\partial(G)$.

Secondly, we need to introduce some *building blocks*, which are essentially components with additional semi-edges, that can be connected together with precisely those semi-edges. The number of semi-edges in our blocks is always even, allowing us to group them into pairs. In all figures depicting a building block, semi-edges belonging to a single pair will point to the same side (left or right). Let G be a graph and B be

a subgraph of G . We use $\iota(B)$, where component B is the building block, to represent the length of the longest induced cycle within the component. Additionally, we define the length of the longest induced path (number of its vertices) in B , traversing through a single semi-edge pair of B as $\iota^\partial(B)$. In other words, $\iota^\partial(B)$ is the number of vertices B can contribute to the longest induced cycle after a combination of building blocks. Each of our building block has an even number of vertices $n \leq 8$ and every vertex has degree 3, making the blocks and their unions cubic. We will now present a list of building blocks. For such building block B , we will present their $\iota(B)$ and $\iota^\partial(B)$. Validity of these values can be easily verified in the accompanying figures.

The smallest block B_A (shown in figure 5.2) is very simple, 2 vertices connected with one edge and each vertex has 2 additional semi-edges, therefore the block has two semi-edge pairs. No cycle can be made in this block, therefore $\iota(B_A) = 0$, but $\iota^\partial(B_A) = 2$ because a cycle can come through one semi-edge from one side, visit a vertex, continue to the other vertex and go back through its semi-edge on the same side, always using only one pair. Obviously, an induced cycle in a cubic graph composed of B_A cannot propagate from the left semi-edge pair to the right (and back), as the edge x_1x_2 would be a chord of such induced cycle. Similar observation can be made about all the building blocks with two semi-edge pairs, hence we refrain from stating this observation for each of them.

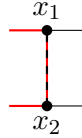


Figure 5.2: A building block B_A contributing 2 vertices to a cycle coming from the left side. The two semi-edge pairs of B_A are both (x_1^*, x_2^*) , the first depicted in red facing left, the other one depicted in black facing right.

Next building block B_B consist of 4 vertices and it looks like a sand clock (shown in figure 5.3). We see 2 parallel paths, both on 2 vertices, plus the top and bottom vertices are cross connected. On each side, there is a semi-edge pair. We can see that $\iota(B_B) = 4$ and $\iota^\partial(B_B) = 3$

The block B_C (see Figure 5.4) is a 6-vertex gadget that internally supports a chordless 5-cycle, $\iota(B_C) = 5$ while exposing exactly two semi-edges on each side. Block B_C can contribute to an induced cycle with a maximum 4 vertices, making $\iota^\partial(B_C) = 4$.

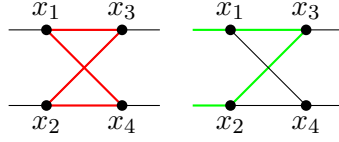


Figure 5.3: Two views of the building block B_B : The first one on the left with its own induced 4-cycle highlighted in red, and the second one on the right with the three vertices contributing to an external cycle lying on the green path.

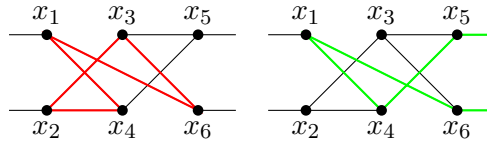


Figure 5.4: Two views of the block B_C : The first one on the left with its own induced 5-cycle highlighted in red, and the second one on the right with the 4 vertices contributing to an external cycle lying on the green path.

The block B_D (see Figure 5.5) can be seen as an extension of block B_C . It contains 8 vertices and, exactly like B_C , can contribute with 4 vertices to an induced cycle, therefore $\iota^\partial(B_D) = 4$. What is not the same, is that B_D internally realizes a chordless 6-cycle instead of 5, having $\iota^\partial(B_D) = 6$. This block also exposes exactly two semi-edges on each side.

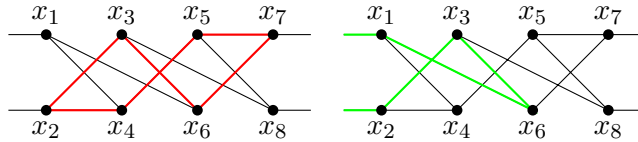


Figure 5.5: Two views of the block B_D : The first one on the left with its internal induced 6-cycle highlighted in red, and the second one on the right with the 4 vertices contributing to an external cycle lying on the green path.

Every building block presented so far can be seen as an *extending* block, having two pairs on semi-edges, meaning we can connect from both sides. Next two blocks will have only 1 pair of these edges each, making them *ending* blocks.

Again we start with the smaller one. Block B_E consists of 4 vertices and, as mentioned, one semi-edge pair (see figure 5.6). The longest cycle inside is a triangle and the number of vertices on the longest induced path is 3. This gives us $\iota(B_E) = 3$ and $\iota^\partial(B_E) = 3$, making this the first and only block with the same number of ι and ι^∂ .

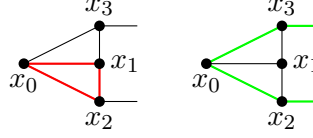


Figure 5.6: Two views of the block B_E : The first one on the left with one of its triangles highlighted in red representing the longest induced cycle within B_E , and the second one on the right with the 3 vertices contributing to an external cycle lying on the green path, here the path can also go through x_1 instead of x_0 .

Now, for the second ending block, also being the last one presented. B_F is a 8-vertex block (shown in figure 5.7) that presents the biggest contrast between ι and ι^∂ . With its own induced cycle of length 6 and contribution to an induced cycle being only 2, making $\iota(B_F) = 6$ and $\iota^\partial(B_F) = 2$.

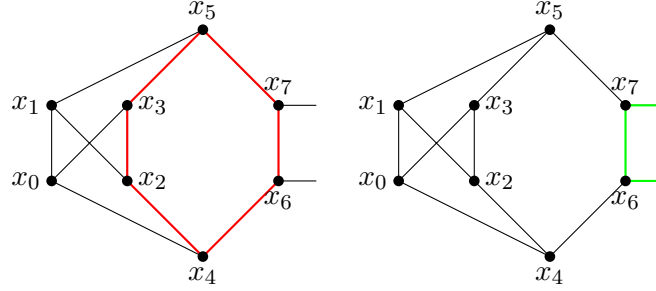


Figure 5.7: Two views of the block B_F : The first one on the left with its internal induced 6-cycle, and the second one on the right with the 2 vertices contributing to an external cycle lying on the green path.

We now define two ways to assemble larger $\kappa = 2$ graphs from our building blocks by connecting their semi-edges. First one is a binary glueing of two blocks via one semi-edge from each, we call it the *fusion* of two blocks B', B'' and denote it by $B' \oplus B''$. The second operation is a ternary glueing, which we call the *join*, denoted $\text{join}(B^{(1)}, B^{(2)}, B^{(3)})$.

Let each block B be specified by its vertex-set $V(B)$, edge-set $E(B)$, and semi-edge set $\partial(B)$. Recall that every semi-edge set is even-sized and partitioned into unordered *semi-edge pairs*.

Definition 1 (Fusion). *Let B' and B'' be two blocks, and let $\{\sigma'_1, \sigma'_2\} \subset \partial(B')$ and $\{\sigma''_1, \sigma''_2\} \subset \partial(B'')$ be one semi-edge pair in each. Their fusion*

$$B' \oplus B''$$

is the block with

$$V(B' \oplus B'') = V(B') \cup V(B''), \quad E(B' \oplus B'') = E(B') \cup E(B'') \cup \{\sigma'_1 \sigma''_1, \sigma'_2 \sigma''_2\},$$

$$\partial(B' \oplus B'') = (\partial(B') \cup \partial(B'')) \setminus \{\sigma'_1, \sigma'_2, \sigma''_1, \sigma''_2\}.$$

That is, we identify each σ'_i with σ''_i (for $i = 1, 2$), creating two new edges and removing those semi-edges from the semi-edge set.

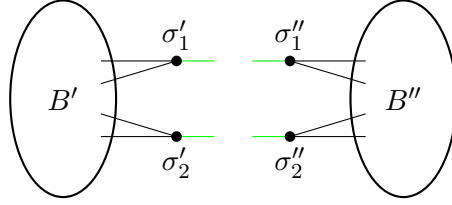


Figure 5.8: Fusion of two building blocks B', B'' . The 4 semi-edges $x_1^*, x_2^*, x_3^*, x_4^*$ depicted in green, connect to make 2 edges x_1x_3, x_2x_4 .

Definition 2 (Join). Let $B^{(1)}, B^{(2)}, B^{(3)}$ be three blocks. Suppose each block's semi-edges are grouped into unordered pairs, and choose

$$\{\alpha_1, \alpha_2\} \subset \partial(B^{(1)}), \quad \{\beta_1, \beta_2\} \subset \partial(B^{(2)}), \quad \{\gamma_1, \gamma_2\} \subset \partial(B^{(3)}).$$

Then $\text{join}(B^{(1)}, B^{(2)}, B^{(3)})$ is the block with

$$V = V(B^{(1)}) \cup V(B^{(2)}) \cup V(B^{(3)}),$$

$$E = E(B^{(1)}) \cup E(B^{(2)}) \cup E(B^{(3)}) \cup \{\alpha_1\beta_2, \beta_1\gamma_2, \gamma_1\alpha_2\},$$

$$\partial = (\partial(B^{(1)}) \cup \partial(B^{(2)}) \cup \partial(B^{(3)})) \setminus \{\alpha_1, \alpha_2, \beta_1, \beta_2, \gamma_1, \gamma_2\}.$$

In other words, we connect $B^{(1)}$ to $B^{(2)}$ by joining α_1 with β_2 , $B^{(2)}$ to $B^{(3)}$ by joining β_1 with γ_2 and $B^{(3)}$ to $B^{(1)}$ by joining γ_1 with α_2 consuming all six chosen semi-edges.

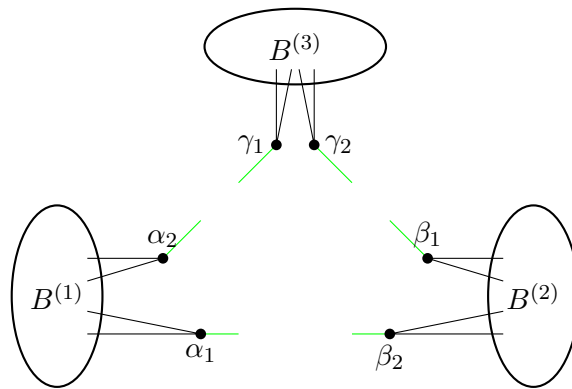


Figure 5.9: Join of 3 building blocks $B^{(A)}, B^{(B)}, B^{(C)}$. The 6 semi-edges $a_0^*, a_1^*, b_0^*, b_1^*, c_0^*, c_1^*$ depicted in green, connect to make 3 edges a_0c_0, c_1b_0, b_1a_1 .

In both cases we simply take the union of vertex-sets and edge-sets, add the new edges formed by pairing semi-edges, and remove those paired semi-edges from the semi-edge sets. This cleanly models “connecting” two or three blocks via their semi-edges.

Both these operations combine the ι^∂ of every block, making an induced cycle of length $\iota^\partial(B') + \iota^\partial(B'')$ for fusion and $\iota^\partial(B^{(1)}) + \iota^\partial(B^{(2)}) + \iota^\partial(B^{(3)})$ for join, therefore

$$\iota(F) = \max(\iota^\partial(B') + \iota^\partial(B''), \iota(B'), \iota(B'')), \quad \iota^\partial(F) = \max(\iota^\partial(B'), \iota^\partial(B'')),$$

where $F = B' \oplus B''$ and

$$\iota(J) = \max(\iota^\partial(B^{(1)}) + \iota^\partial(B^{(2)}) + \iota^\partial(B^{(3)}), \iota(B^{(1)}), \iota(B^{(2)}), \iota(B^{(3)}))$$

$$\iota^\partial(J) = \max(\iota^\partial(B^{(1)}), \iota^\partial(B^{(2)}), \iota^\partial(B^{(3)}))$$

where $J = \text{join}(B^{(1)}, B^{(2)}, B^{(3)})$.

Having developed our repertoire of building-blocks and the two ways to assemble them (*fusion* and *join*), we now turn to two concrete subclasses of bridgeless cubic graphs, classified by their connectivity κ , girth g , and the length ι of a longest induced cycle. Throughout what follows we will denote such graphs simply as (κ, g, ι) -graphs.

In Sections 5.2.1 and 5.2.2 we focus on the case $\kappa = 2$. First, we analyze all $(2, 3, 5)$ -graphs, identifying precisely which blocks and fusion patterns generate every such graph, and thereby showing that $\iota = 5$. Next, we study the $(2, 4, 6)$ -graphs, determining the exact blocks and sequences of fusion and join required to build them, and establishing that $\iota = 6$. These two subsections together illustrate how our block glueing framework yields a complete structural classification of all cubic graphs by their (κ, g, ι) parameters.

5.2.1 (2,3,5)-graphs

In this subsection we show that every bridgeless cubic graph G with connectivity $\kappa = 2$, girth $g = 3$, and longest induced cycle $\iota(G) = 5$ can be built by fusing copies of the end-block B_E and extension blocks B_A and B_B . No other blocks or the ternary join operation are needed (and in fact would force $\iota > 5$).

Lemma 7. *Let G be a connected cubic graph obtained by fusing end-blocks B_E and extension blocks B_A, B_B in the pattern*

$$G = B_E \oplus B_A^{a_0} \oplus (B_B \oplus B_A^{a_1}) \oplus \cdots \oplus (B_B \oplus B_A^{a_r}) \oplus B_E,$$

for some $r \geq 0$ and integers $a_0, a_1, \dots, a_r \geq 1$, where

- B_E is the end-block with $\iota(B_E) = 3$,
- B_A and B_B are the only extension blocks with $\iota^\partial(B_A) = 2$ and $\iota^\partial(B_B) = 3$,
- no two B_B s are adjacent (since $3 + 3 > 5$), nor can B_B fuse directly to B_E (since $3 + 3 > 5$).

Then G is a bridgeless cubic graph with $\kappa(G) = 2$, $g(G) = 3$, and $\iota(G) = 5$.

Proof. Let

$$G = B_E \oplus B_A^{a_0} \oplus (B_B \oplus B_A^{a_1}) \oplus \cdots \oplus (B_B \oplus B_A^{a_r}) \oplus B_E$$

be the given fusion chain.

Cubicity and bridgelessness. Each block B_E, B_A, B_B is cubic and bridgeless, and fusing along semi-edge pairs removes two semi-edges from each block and replaces them by two new edges. Thus every vertex still has degree three, and no bridge can be created at a fusion, so G is a bridgeless cubic graph.

Girth $g(G) = 3$. Each end-block B_E contains a triangle and no block has a smaller cycle, so G has girth exactly 3.

Induced-cycle length $\iota(G) = 5$. Fusing two blocks $B' \oplus B''$ creates an induced cycle of length $\iota^\partial(B') + \iota^\partial(B'')$. Since

$$\iota^\partial(B_A) = 2, \quad \iota^\partial(B_B) = 3, \quad \iota^\partial(B_E) = 3,$$

the only fusions that achieve exactly 5 are

$$B_A \oplus B_B : 2 + 3 = 5, \quad B_A \oplus B_E : 2 + 3 = 5 \quad (\text{or symmetrically}).$$

No other fusion produces a longer induced cycle, and no ternary join is used. Hence the maximum induced-cycle length in G is 5.

Connectivity $\kappa(G) = 2$. Each fusion along a semi-edge pair identifies two vertices, which form a 2-vertex separator between the two fused subgraphs. Removing those two vertices disconnects G at that fusion point, so $\kappa(G) \leq 2$. On the other hand, each block is 2-connected and the chain begins and ends with B_E , so removing any single vertex cannot disconnect G . Therefore $\kappa(G) = 2$.

Combining these observations, G is bridgeless, cubic, 2-connected, has girth 3, and its longest induced cycle has length 5, as claimed. $\square$

Based on exhaustive enumeration for $n \leq 24$ and the tight correspondence between our fusion patterns and the extremal examples in Section 5.1, we conjecture that the converse of Lemma 7 holds:

Conjecture 2. *Let G be a bridgeless cubic graph with $\kappa(G) = 2$, $g(G) = 3$, and $\iota(G) = 5$. Then G can be assembled by fusing end-blocks B_E and extension blocks B_A, B_B in the pattern described in Lemma 7.*

5.2.2 (2,4,6)-graphs

In the (2,4,6) case the girth $g = 4$ excludes triangle-blocks, so B_E is unavailable and the only end-block is B_F (with $\iota(B_F) = 6$). All other blocks B_A, B_B, B_C, B_D have girth

at least 4 and $\iota \leq 6$, so they serve as extenders, used only via fusion or the ternary join.

Lemma 8. *Let G be a connected cubic graph with $\partial(G) = \emptyset$ obtained by an assembly of blocks using*

$$B' \oplus B'' \quad \text{and} \quad \text{join}(B^{(1)}, B^{(2)}, B^{(3)})$$

with end-blocks B_F and extenders B_A, B_B, B_C, B_D , subject to: - any fusion $B' \oplus B''$ must satisfy $\iota^\partial(B') + \iota^\partial(B'') \leq 6$, and - any join $\text{join}(X, Y, Z)$ must have $\iota^\partial(X) = \iota^\partial(Y) = \iota^\partial(Z) = 2$. Then G is bridgeless, cubic, $\kappa(G) = 2$, $g(G) = 4$, and $\iota(G) = 6$.

Proof. Each block is cubic and bridgeless, and fusion/join simply replaces semi-edges by edges, preserving cubicity and bridgelessness. Since all used blocks have a girth of at least 4, the resulting graph also has $g(G) = 4$.

Fusions contribute induced cycles of length $\iota^\partial(B') + \iota^\partial(B'') \leq 6$, and by assumption, any pair exceeding 6 is forbidden. Joins always use three blocks of induced cycle-contribution 2, so each join inserts a 6-cycle. Also some of the blocks have cycles of length 6 within them. Therefore $\iota(G) = 6$.

Finally, each fusion or join along a semi-edge pair creates a 2-vertex separator in the chain. Removing one vertex never disconnects a 2-connected block, so $\kappa(G) \geq 2$, while removing both vertices of any fused pair disconnects the chain, so $\kappa(G) = 2$. $\square$

Empirical validation on all cubic graphs up to $n = 24$ shows that every observed $(2, 4, 6)$ -graph admits an assembly via the fusion and join rules of Lemma 8. This leads us to the following converse conjecture.

Conjecture 3. *Every bridgeless cubic graph G with $\kappa(G) = 2$, $g(G) = 4$, and $\iota(G) = 6$ can be assembled from end-blocks B_F and extenders B_A, B_B, B_C, B_D via the fusion and join operations of Lemma 8.*

5.3 Planar Cubic Graphs

It is worth noting that every $(2, 3, 5)$ -graph admits a planar drawing (though embedding the B_B block can be somewhat awkward). However, since we restrict our analysis to 3-connected cubic planar graphs, the $\kappa = 2$ families—including the $(2, 3, 5)$ class—do not appear in Figure 5.10.

We used `plantri`[2] to generate all 3-connected graphs on $8 \leq n \leq 28$ vertices with girth $g \in \{3, 4, 5\}$. For each graph G , we computed $\iota(G)$ and recorded, for each (n, g) pair, the minimum and maximum values observed.

As Figure 5.10 shows, planarity substantially raises the floor on $\iota(G)$. In particular:

- For $n = 8$ and $n = 10$, all planar cubics in our sample share the same $\iota(G)$ —namely 6 and 7, respectively—so there is no variability between girth classes. However, we need

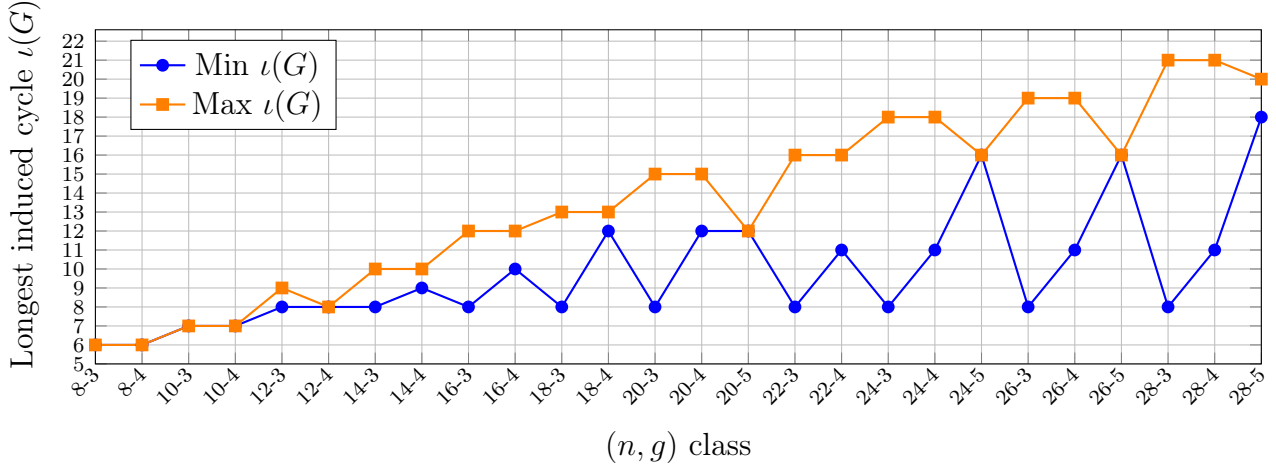


Figure 5.10: Minimum and maximum values of the longest induced cycles $\iota(G)$ across each (n, g) subclass of planar graphs.

to point out that in both classes with girth 4, only one graph was contained in the class.

- Starting at $n = 12$, the smallest observed $\iota(G)$ in the girth-3 subclass jumps to 8 and never visits 5. In other words, the “spiked-ladder” obstructions ($\iota = 5$) found in the general cubic case do not embed in planar cubic graphs given by plantri.

- For each $n \geq 12$, the maxima in the girth-3 and girth-4 classes coincide: forbidding triangles raises the minimum allowed cycle length but does not further restrict the maximum once $g \leq 4$.

These data indicate that the planarity constraint alone eliminates those small-cycle obstructions and enforces a strictly larger lower bound on $\iota(G)$. Understanding exactly which planar-forbidden minors or structural properties preclude the spiked-ladder configurations remains an intriguing open problem.

5.4 Open Conjectures

The computational experiments of Chapter 5 reveal striking patterns in how connectivity, girth, and planarity constrain the length of the longest induced cycles in cubic graphs. While our block-decomposition results account for the extremal $(2, 3, 5)$ and $(2, 4, 6)$ families, a broader theoretical framework remains elusive. Below we isolate the most compelling observations as precise conjectures, in hopes of guiding future work toward a unified understanding of induced-cycle extremality.

These first two conjectures propose exact lower bounds on $\iota(G)$ determined solely by the presence of small cycles. They suggest that triangles enforce a minimum induced-cycle length of 5, while rectangles raise that floor to 6.

Conjecture 4 (Girth-3 Minimum). *In any bridgeless cubic graph G of order $n \geq 8$, if G contains a triangle (girth 3), then*

$$\iota(G) \geq 5.$$

Conjecture 5 (Girth-4 Minimum). *In any bridgeless cubic graph G of order $n \geq 8$, if G contains a rectangle (girth 4), then*

$$\iota(G) \geq 6.$$

The next conjecture elevates this phenomenon further in the planar setting, where global embedding constraints appear to force even longer induced cycles.

Conjecture 6 (Planar Elevation). *Let G be a 3-connected cubic planar graph of order $n \geq 12$. Then*

$$\iota(G) \geq 8.$$

In particular, no planar cubic graph on $n \geq 12$ vertices admits $\iota(G) \leq 7$.

Finally, we turn from lower-bound phenomena to the question of upper-bound tightness, conjecturing that the general $\lfloor 3n/4 \rfloor$ limit is attained infinitely often.

Conjecture 7 (Tightness of the $\frac{3n}{4}$ Bound). *For every integer $n \geq 8$, there exists a bridgeless cubic graph G of order n such that*

$$\iota(G) = \left\lfloor \frac{3n}{4} \right\rfloor.$$

In other words, the bound of Lemma 1 cannot be improved asymptotically.

These conjectures distil the empirical phenomena observed in section 5.1 and 5.3. Proving (or refuting) them would deepen our understanding of how local structural constraints—such as small girth or planarity—govern the existence of long chord-free cycles in cubic graphs.

Conclusion

We have designed an efficient algorithm for computing the length of the longest induced cycles in cubic graphs, leveraging the 3-regular structure to enforce chord-freeness via a dynamic set and applying strong pruning to manage the search. Theoretical analysis guarantees correctness, and empirical tests on diverse graphs confirm that the algorithm consistently identifies optimal cycles in milliseconds on problems where naive methods take orders of magnitude longer.

Our experiments also reveal structural phenomena ripe for further investigation: the persistence of the linear upper bound across rich graph subclasses, the elimination of small induced cycles in planar graphs, and conjectured monotonicity relationships between connectivity, girth, and minimal induced-cycle length. Proving these conjectures could deepen our understanding of how local constraints govern induced-cycle structure in sparse graphs. Another promising direction is to investigate how increased cyclic connectivity influences the existence and length of induced cycles in cubic graphs.

Moreover, the block-gluing framework developed for the $(2, 3, 5)$ - and $(2, 4, 6)$ -graphs not only explains why these subclasses attain their minimal induced-cycle lengths, but also suggests a modular approach to analyzing other connectivity–girth families. Future structural work might extend this decomposition perspective to $\kappa = 3$ or higher-girth classes, further bridging algorithm and theory.

Future work may also extend our approach to larger instances through additional reductions or parallelization, adapt the $\mathcal{S}$ -set framework to weighted or directed graphs, and explore analogous problems in higher-regularity or other combinatorial settings. Overall, this thesis lays a foundation—both algorithmic and structural—for efficient computation and theoretical analysis of long induced cycles in low-degree graphs.

Bibliography

- [1] Reinhard Diestel. *Graph Theory*. Springer, 3rd edition, 2005. Electronic Edition.
- [2] Brendan McKay Gunnar Brinkmann. Planar graphs (plantri). <http://combos.org/plantri>. Accessed: 2025-05-21.
- [3] Michael A. Henning, Felix Joos, Christian Löwenstein, and Thomas Sasse. Induced cycles in graphs, 2014.
- [4] M. Meringer. Regular graphs (genreg). <https://www.mathe2.uni-bayreuth.de/markus/reggraphs.html>. Accessed: 2025-05-21.